

## **INTRODUÇÃO À ANÁLISE DE SISTEMAS**

Lei de Moore (Moore, 1965), proposta por um dos fundadores da Intel, Gordon Moore, que observou que a quantidade de transistores em um processador duplicava a cada 12 meses.

alguns dos reflexos da Lei de Moore:

- Diminuição do custo de produção: com o baixo custo de produção dos computadores, foi possível reduzir o preço final e, conseqüentemente, houve um aumento nas vendas e na produção em escala dos equipamentos.
- Diminuição do consumo de energia: os computadores passaram a consumir menos energia, tornando-se cada vez mais atrativos a um público cada vez mais crescente.
- Computadores menores: com componentes menores, os computadores também reduziram de tamanho. Passamos a não ter mais a necessidade de um local físico específico e especial para o computador.
- Computadores mais confiáveis: com um maior número de transistores em uma pastilha, diminuiu a necessidade de conexões entre pastilhas.

### **Sistemas de informação x software**

Sistema de informação é a composição de informações (ou dados), pessoas, processos e tecnologias que tem por objetivo apoiar ou melhorar o processo de negócio de uma corporação.

Sistemas de informação tem foco na informação gerada em um processo organizacional, de tal forma a originar valor a esse processo.

Tecnologia de sistemas de informação, estamos falando de toda plataforma tecnológica que dá suporte ao processo de negócio, como infraestrutura de rede, servidores, computadores, sistemas operacionais e sistemas de software

Na plataforma tecnológica, deveríamos nos atentar a detalhes como como e onde essa informação seria armazenada e qual o melhor projeto de rede para suportar o tráfego dessa informação.

Projetar um sistema de software compreende termos a noção de que ele é um componente de um sistema de informação

Diversas foram as abordagens e técnicas criadas para modelar sistemas de software até chegarmos à abordagem foco deste livro: orientação a objetos. Por ora, entenda que a orientação a objetos é uma forma de se projetar um sistema de software.

Um sistema de software também pode ser identificado como o conjunto de vários componentes que, quando executados, produzem um resultado previamente desejado.

A Engenharia de Software tem por objetivo apoiar o desenvolvimento de software fornecendo técnicas para especificar, projetar, manter e gerir um sistema

Ela é baseada em três pilares:

- Métodos: provêm um conjunto de atividades com ênfase no como fazer para desenvolver um sistema de software.
- Ferramentas: mecanismos que dão suporte à execução das atividades descritas no método.
- Processos: proveem o elo entre métodos e ferramentas. Definem o quando e onde fazer, indicando a sequência de execução das atividades, quais ferramentas utilizar e quem deve ser o responsável pela atividade.

## **PROJETO DE DESENVOLVIMENTO**

### **Processo de desenvolvimento de software**

São um conjunto de atividades executadas em uma determinada sequência. Esse conjunto de atividades também pode ser chamado de etapas da Engenharia de Software

**Especificação de software** - Fase na qual são definidas e especificadas as funcionalidades do software, seus requisitos e suas regras de execução

**Projeto e implementação de software** - Fase na qual os requisitos são transformados em sistema de software. Quando são projetados e construídos os componentes de software e suas integrações

**Validação de software** - Fase na qual o software é validado frente aos requisitos especificados

**Evolução de software** - Fase na qual são traçadas estratégias para que o sistema de software possa ser mantido e evoluído na medida

**Processo de desenvolvimento de software. Cada atividade possui:**

- Pré e pós-condições: qual é o cenário pré e pós-execução de uma atividade. Por exemplo: para validação de um software, é necessário que este esteja implementado, após execução da validação, espera-se que o software esteja validado.

- Papéis: quem é o responsável pela execução da tarefa. Por exemplo: o analista de testes é responsável pela validação.

- Produtos: também chamados de artefatos, é o que é produzido como saída de uma determinada atividade. Por exemplo: espera-se como artefato de saída da atividade de especificação de software um documento contendo a descrição detalhada de todas as necessidades do negócio.

**Modelos de processo de software**

**Modelo cascata**

Também chamado de waterfall ou também citado na literatura como ciclo de vida clássico, o modelo cascata é composto de cinco atividades: Análise de requisitos, Projeto, Codificação, Testes, Implantação e manutenção.

No modelo cascata, as atividades são executadas de forma sequencial. Assim, uma atividade não é iniciada até que sua predecessora seja completamente finalizada. Isso nos permite afirmar, por exemplo, que a construção não é iniciada até que seja finalizada a arquitetura do sistema de software na fase de projeto, que, por sua vez, não é iniciada até que todos os requisitos sejam levantados, documentados e aprovados pelo usuário, é muito eficaz em projetos nos quais temos o escopo e as necessidades bem definidas e com poucas chances de mudança

## **Modelo incremental**

O modelo incremental, também chamado de iterativo. No modelo iterativo, as atividades e a sequência de execução são exatamente as mesmas do modelo cascata. A diferença é que no modelo incremental não é necessário que todos os requisitos estejam mapeados para se iniciar o ciclo de desenvolvimento

O ciclo é iniciado a partir de blocos de requisitos e não a partir de todos os requisitos, como pregado no modelo clássico. A partir daí, a metodologia é a mesma, ou seja, cada atividade somente é iniciada depois que a predecessora é finalizada e validada. Cada bloco de requisitos, quando implantado, corresponde a uma parte do sistema de software, também chamado de incremento. O ciclo é repetido tantas vezes quanto necessário

## **Prototipagem**

Em Engenharia de Software podemos pensar que o protótipo é a primeira versão de um sistema de software.

Esse protótipo de software tem como objetivo a prova de conceitos. Verificar e demonstrar se os requisitos mapeados estão de acordo com a necessidade do usuário ou validar uma solução de projeto.

**O modelo de prototipagem possui quatro atividades:** • planejamento; • definição das funcionalidades; • construção; • validação.

Quando usar a prototipação?

- Se nosso usuário tem uma boa noção, mas não tem, no momento, uma visão detalhada do que precisa ser feito.
- O cliente precisa ter a total noção de que o protótipo não é o sistema de software.
- Toda a equipe de desenvolvimento também precisa ter a total noção que o protótipo não é o sistema de software

Protótipo de baixa fidelização, que dá ênfase na interação com o usuário e na estrutura geral do sistema de software, não se preocupando em fazer protótipos que produzissem uma imagem real do sistema.

## **Modelo espiral**

Tem como raiz o modelo iterativo incremental e como preocupação central a mitigação de riscos.

A diferença fundamental do modelo espiral para os outros modelos que discutimos até agora é que cada ciclo completo, ou cada iteração, não produz ou implementa um sistema ou uma parte do sistema de software.

O modelo espiral é composto de quatro fases: • Definição dos objetivos. • Mitigação dos riscos. • Desenvolvimento do produto. • Planejamento da próxima fase.

## **Processo unificado**

Processo Unificado, ou UP (Unified Process), tem em seu fundamento a estrutura do modelo iterativo incremental, quatro fases:

- **Iniciação:** nesta fase é definido o escopo do projeto.
- **Elaboração:** o objetivo desta fase é detalhar o escopo, produzindo o modelo de requisitos do sistema. Nesta fase pode-se produzir o plano de projeto, ou plano de desenvolvimento, que consiste no mapeamento dos requisitos, as soluções de software.
- **Construção:** esta fase está associada ao desenvolvimento do projeto, que consiste em projetar, codificar e testar o sistema de software.
- **Transição:** nesta fase, o sistema de software é entregue para o usuário final, sendo comum que existam correções e adaptações a serem feitas.

**UP é baseado em três pilares:** • Dirigido por casos de uso. • Centrado na arquitetura. • Iterativo e Incremental.

**O RUP é um modelo de processos**, desenvolvido pela Rational, que associou o Modelo de Processo Unificado às melhores práticas da Engenharia de Software, que são: • Desenvolva iterativamente. • Gerencie requisitos. • Use

arquitetura de componentes. • Modele visualmente. • Verifique continuamente a qualidade. • Gerencie mudanças.

### **Processo x pessoas**

- O que será produzido: qual é o produto ou o artefato do processo.
- Como: atividade, ou sequencia de atividades, a serem realizadas, em uma determinada sequência para produção do artefato do processo.
- Quando: definição de qual momento o artefato será produzido, definição das pré-condições e pós-condições.
- Quem: indicação do responsável pela atividade.

Papel: “define o comportamento e as responsabilidades de um indivíduo ou de um conjunto de indivíduos que trabalham juntos como uma equipe, no contexto de uma organização de engenharia de software”

No caso do RUP, os papéis são sistematicamente definidos, por exemplo: o analista de sistemas é responsável, entre outras coisas, por desenvolver o plano de gerenciamento de requisitos e o arquiteto de software executa, entre outras atividades, a estruturação do modelo de implantação.

### **Gerente de projetos**

Profissional responsável por gerir o projeto, que compreende, entre muitas coisas, coordenar e acompanhar as atividades do projeto e alocar as pessoas para as funções determinadas.

### **Analista**

É responsável por captar as necessidades do negócio e transformá-las em requisitos do sistema de software.

### **Projetista**

Responsável por produzir uma solução computacional para os problemas identificados. Por exemplo: um projetista de interface gráfica do usuário, que é especialista em fornecer soluções para interfaces padrão para sistemas operacionais da Apple ou da Microsoft.

## **Arquiteto de software**

Responsável pela elaboração da arquitetura do sistema. Como e quais serão os componentes internos do sistema de software e como esses componentes irão se comunicar para atender a uma determinada necessidade.

## **Desenvolvedores**

Também chamados de programadores, os desenvolvedores têm a responsabilidade de implementar o projeto.

## **Clientes**

O usuário é parte fundamental do processo de desenvolvimento, ele precisa estar envolvido e com foco desde o momento do levantamento de requisitos até a validação.

## **Avaliadores de qualidade**

São responsáveis por garantir que o sistema de software atenda a todas as necessidades do negócio.

## **O paradigma da orientação a objetos**

-paradigma: “Um paradigma é um conjunto de regras que estabelecem fronteiras entre o que é certo e errado, entre o que é verdadeiro e o que é falso, entre o que se deve fazer e o que não se deve fazer [...] Ele funciona como um conjunto de categorias que define e orienta o comportamento das pessoas”

O paradigma da orientação a objetos é uma forma de se desenvolver um sistema de software que o enxerga como um conjunto de componentes que interagem entre si para resolver um determinado problema. A cada componente, dá-se o nome de objeto.

Em orientação a objetos, cada objeto também possui características, denominadas atributos, e a ação a ser executada por esse objeto, que tem o nome de método.

O paradigma da orientação a objetos é baseado nos seguintes pilares: • classes; • objeto; • abstração; • encapsulamento; • herança; • polimorfismo; • ligação, mensagem.

### **Classe**

Classe de objetos pode ser definida como um grupo de objetos com as mesmas propriedades (atributos), comportamentos (operações), relacionamentos e semântica.

### **Objeto**

objeto é um elemento do mundo real, que possui relevância para solução de um determinado problema e que esse objeto possui características e executa determinadas ações (métodos). Esse conjunto de informações, também chamado de atributos, identifica um objeto em uma determinada classe de objetos.

### **Abstração**

O conceito de abstração está ligado à nossa capacidade, enquanto analistas, de resolver problemas, de selecionar determinados aspectos do problema e isolar o que é importante do que não é para um determinado propósito. Abstração é dar ênfase àquilo que é essencial.

### **Encapsulamento**

Encapsulamento significa deixar visível apenas o que é necessário para a comunicação entre dois objetos (Ex: detalhes da implementação ou a lógica algorítmica de um método).

### **Herança**

Objetos que possuem características semelhantes, mas que estão em classes diferentes



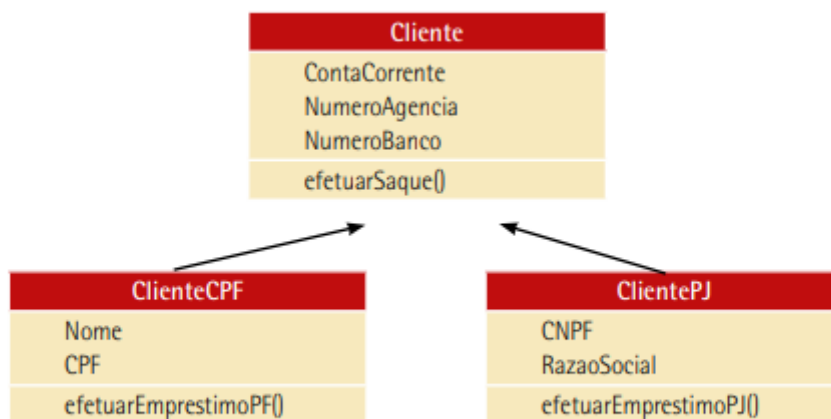


Figura 7 – Exemplo de herança

### Dois tipos de herança:

- Simples: quando uma classe filha herda características de apenas uma classe mãe.
- Composta: quando uma classe filha herda características de mais de uma classe mãe.

### Polimorfismo

Polimorfismo é quando um objeto tem um comportamento diferente para a mesma ação.

Em orientação a objetos, polimorfismo significa que um método definido na classe mãe é redefinido, tem uma nova implementação, na classe filha, sendo obrigatória a mesma assinatura para o método.

### Ligação e mensagem

Mensagem é um meio de comunicação entre objetos com o objetivo de ativar um processamento. Para que um objeto execute um método.

Se um objeto envia uma mensagem para outro, eles possuem uma ligação, chamada de associação simples, ou seja, uma ligação que acontece apenas em um determinado momento.

Pode acontecer que um objeto tenha em sua composição outro objeto. Neste caso existe uma relação todo-parte.

Quando o objeto que compõe outro não existe fora desse contexto, dizemos que existe uma associação do tipo composição.

Quando o objeto que compõe o outro existe fora do contexto, dizemos que há uma associação do tipo agregação.

### **A linguagem de modelagem unificada**

A Linguagem de Modelagem Unificada, ou Unified Modeling Language, conhecida como UML, é uma linguagem para modelar sistemas orientados a objetos.

- **Regra de sintaxe:** cada elemento deve ser desenhado, representado, seguindo um padrão.
- **Regra semântica:** define o significado de cada elemento dentro de um contexto.

UML é uma ferramenta que dá suporte a todo o processo, desde a análise de requisitos, passando pelo projeto, construção e até a validação, um sistema de software pode ser dividido em cinco visões:

- **Caso de Uso:** representa o sistema de um ponto de vista externo, como ele interage com agentes externos, como usuários ou outros sistemas.
- **Projeto:** representa a arquitetura do sistema de software.
- **Implementação:** representa os componentes e subsistemas.
- **Implantação:** representa a distribuição física do sistema e seus componentes e como estes irão se comunicar.
- **Processo:** dá ênfase na representação do paralelismo e sincronização dos componentes do sistema.

Um diagrama UML é uma composição de elementos gráficos com o objetivo de representar alguma perspectiva do sistema.

## **ENGENHARIA DE REQUISITOS E MODELOS DE CASOS DE USO**

## **Introdução à Engenharia de Requisitos**

Engenharia é um conjunto de métodos, procedimentos e ferramentas que tem por objetivo resolver um determinado problema.

Requisitos são serviços que um sistema deve prestar e suas restrições de funcionamento, refletem as necessidades do cliente.

O mais interessado nos requisitos é o cliente e o usuário, mas eles possuem papéis diferentes. O cliente é o contratante do projeto, o usuário é quem vai, efetivamente, usar o sistema, quem detém o maior conhecimento do negócio, tbm são considerados interessados pelos requisitos: analista de sistemas, desenvolvedores, arquitetos e gerentes de projeto.

### **Classificação de requisitos**

- **Requisitos de usuário:** “declarações em linguagem natural com diagramas de quais serviços o sistema deverá fornecer a seus usuários e as restrições com as quais ele deve operar”. (direcionados aos clientes, usuários, gerentes de projeto e arquitetos de sistema)

- **Requisitos de sistema:** “descrições mais detalhadas das funções, serviços e restrições operacionais do sistema de software”. (direcionados a usuários, arquitetos de sistema e desenvolvedores, pois podem definir uma sequência de implementação)

requisitos não devem contemplar informações de gestão do projeto, detalhes da arquitetura, projeto ou implementação, bem como informações sobre como o sistema será validado. Os requisitos de software devem dar ênfase somente no comportamento do software que será construído e mantido.

### **Requisitos funcionais**

Requisitos funcionais (RF) descrevem o comportamento esperado de um sistema de software, explicita o que o sistema deve fazer e idealmente o que o sistema não deve fazer, requisito funcional é uma descrição detalhada da interação entre o software e o usuário.

### **Requisitos não funcionais**

Requisitos não funcionais (RNF) descrevem restrições sobre os serviços oferecidos pelo sistema de software

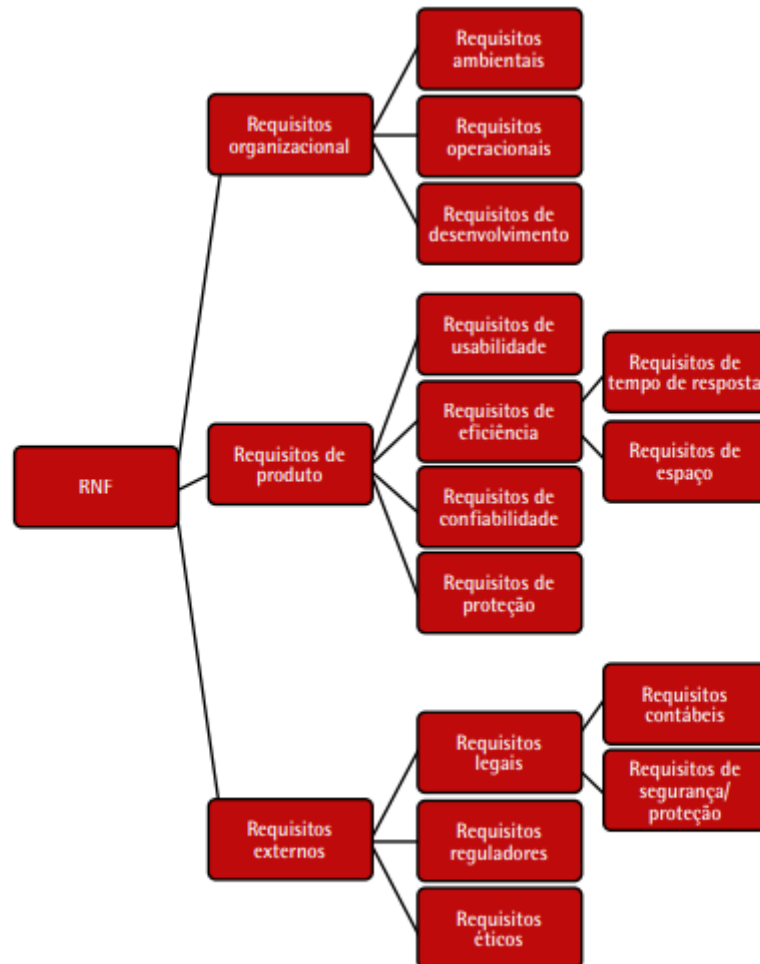


Figura 11 – Requisitos não funcionais

• **Requisitos Organizacionais:** provenientes da organização do cliente, como a padronização de uma linguagem de desenvolvimento. • **Requisitos de Produto:** são aqueles que restringem o comportamento do sistema de software, como o espaço em disco que ele irá ocupar. • **Requisitos Externos:** são requisitos de fora da fronteira do sistema de software, como requisitos legais, de cumprimento da legislação.

**Norma ISO/IEC 25010:2011** => descreve seis características que definem a qualidade de software: funcionalidade, confiabilidade, usabilidade, eficiência, manutenibilidade e portabilidade. (atributos de qualidade)

| Atributo         | Descrição                                                                                                                                                                        |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Funcionalidade   | Está ligado à capacidade do sistema de <i>software</i> de prover funcionalidades que atendam às necessidades explícitas e implícitas quando usado sob as condições especificadas |
| Confiabilidade   | Está ligado à capacidade do sistema de <i>software</i> de manter um determinado nível de desempenho quando usado sob as condições especificadas                                  |
| Usabilidade      | Está ligado à capacidade do sistema de <i>software</i> de auxiliar os usuários na realização de suas tarefas, de maneira produtiva (Rocha; Baranauskas, 2003)                    |
| Eficiência       | Está ligado à capacidade do sistema de <i>software</i> de prover desempenho apropriado, relativo à quantidade de recursos utilizados                                             |
| Manutenibilidade | Está ligado à capacidade do sistema de <i>software</i> de ser modificado, sendo que essa modificação pode ser uma correção, melhoria ou adaptação                                |
| Portabilidade    | Está ligado à capacidade do sistema de <i>software</i> de ser portátil entre plataformas de ambientes                                                                            |

## Processo de Engenharia de Requisitos

Engenharia de Requisitos é um conjunto de métodos, procedimentos e ferramentas que tem por objetivo descobrir, analisar, documentar, verificar e validar esses requisitos.

O processo de engenharia de requisitos tem como objetivo obter requisitos definidos, especificados e modelos de sistema a partir de fontes de requisitos

As fontes de requisitos podem ser:

- Sistemas de informações existentes.
- Necessidade dos interessados (stakeholders).
- Padrões da organização.
- Informações do Domínio.
- Regulamentos (ou legislações).

Cinco atividades principais:

-elicitação (descoberta dos requisitos a partir das fontes de requisitos),

-análise e negociação ( os requisitos são analisados e os conflitos resolvidos por meio da negociação com os interessados. Nessa atividade é produzido o Modelo de Sistema com os requisitos do usuário.),

-documentação (os requisitos são detalhados de tal modo que permitam a realização das próximas atividades do desenvolvimento. É produzida a especificação dos requisitos contendo os requisitos de usuário e de sistema.),

-validação (os requisitos são validados de acordo com os critérios definidos na documentação dos requisitos.)

-gerenciamento (atividade de gerenciamento de requisitos, como se pode notar, não está representada na figura, isso por que a meta da gerência de requisitos é o controle da mudança dos requisitos ao longo do processo da engenharia de requisitos.)

### **Elicitação de requisitos**

A elicitación de requisitos tem como objetivo a descoberta dos requisitos e das fronteiras do sistema, que determinam o escopo do projeto.

### **Entrevistas**

Trata-se basicamente da conversa direcionada a um objetivo: o de obter informações sobre um problema de um entrevistado.

### **Cenários**

Nesta técnica é montado um cenário de interações entre o usuário e o sistema para que ele execute uma determinada tarefa.

### **Protótipos**

Um protótipo é uma versão não operacional do sistema de software que permite esclarecer eventuais requisitos.

### **Reuniões facilitadas**

brainstorming, no qual um grupo de usuários é reunido para debater a respeito de um determinado requisito. É importante que essa reunião ocorra com um mediador, que define o objetivo a ser tratado, capte as principais ideias

### **Observação**

Os analistas aprendem sobre as tarefas do usuário por imersão em seu e observando como eles interagem com seu software e uns com os outros.

### **Análise de documentos**

Os requisitos são elicitados a partir de documentos da organização. Um exemplo muito próximo à nossa realidade são os editais de licitação para desenvolvimento de software

### **Análise de requisitos**

Com os requisitos identificados, é necessário explicitar as informações obtidas na elicitação. Esse processo auxilia no entendimento dos requisitos elicitados, eliminando possíveis ambiguidades e problemas que possam ter passado despercebidos pelo processo de elicitação.

Explicitar os requisitos significa especificar as características operacionais do software, descrever suas restrições e suas interações com outros sistemas a fim de estabelecer um modelo conceitual do problema.

### **Documentação de requisitos**

A documentação dos requisitos tem por objetivo formalizar os requisitos e modelos definidos nas etapas anteriores.

- Documento de definição de sistema: este documento contém os requisitos de sistema, ou seja, requisitos em alto nível de abstração na perspectiva do domínio. Podendo incluir requisitos não funcionais e modelos conceituais.

- Especificação de requisitos de sistema: este documento contém os requisitos na visão sistêmica, ou seja, não somente os requisitos do sistema de software, mas tudo que o circunda.

- Especificação de requisitos de software: são os requisitos na visão do sistema de software. Esse documento pode ser utilizado como base para o desenvolvimento e para as futuras validações.

### **Validação de requisitos**

O objetivo do processo de validação é assegurar que o trabalho de elicitação, análise e documentação dos requisitos está consistente com o domínio do projeto.

### **Gerenciamento de requisitos**

Inevitavelmente, requisitos podem ser incompletos e inconsistentes, bem como novos requisitos podem surgir.

A meta da gerência de requisitos é o controle da mudança dos requisitos ao longo do processo de engenharia de requisitos.

Gerência de requisitos tem mais dois objetivos: • Gerenciar o relacionamento entre requisitos. • Gerenciar o relacionamento entre requisitos e os documentos produzidos durante todo o projeto.

Dois dos modelos mais difundidos atualmente são: CMMI (Capability Maturity Model for Software) e o MPS.BR (Melhoria de Processo do Software Brasileiro).

### **Controle de versão**

É importante que tenhamos sob controle, por qualquer mecanismo que seja, quais documentos fazem parte de uma versão base, uma vez que, a partir da mudança de um requisito, é gerada uma nova versão. Sem um controle de versão, seria praticamente impossível garantir o trabalho em equipe e manter o histórico de evolução do software.

### **Rastreabilidade**

Rastreabilidade especifica as dependências de um requisito para com outro e de um requisito com cada artefato gerado ao longo do projeto. Quando falamos de artefatos, não estamos falando apenas dos gerados durante a engenharia de requisitos, mas sim em todo o projeto, como qual requisito está relacionado à qual solução de software

Trilha de auditoria => permite identificar tudo que pode ser afetado no caso da mudança de um requisito

Matriz de rastreabilidade => objetivo cruzar um requisito com outro dependente ou um requisito com um artefato.



Existem diversas matrizes de requisitos. Entre as mais utilizadas

- Matriz que indica o relacionamento requisito x requisito.
- Matriz que indica o relacionamento requisito x fonte do requisito.
- Matriz que indica o relacionamento requisito x subsistemas, que descreve quais subsistemas atendem àquele requisito.

- Matriz que indica o relacionamento entre um requisito elicitado e um artefato produzido no processo da Engenharia de Software

## **MODELAGEM DE CASOS DE USO E MODELAGEM DE PROCESSO DE NEGÓCIO**

### **Modelagem de casos de uso**

Engenharia de requisitos é um conjunto de métodos, procedimentos e ferramentas que tem por objetivo descobrir, analisar, documentar, verificar e validar esses requisitos.

A análise de requisitos tem como objetivo explicitar os requisitos elicitados, o que significa especificar as características operacionais do software.

“modelo de casos de uso é uma representação das funcionalidades externamente observáveis do sistema e dos elementos externos ao sistema que interagem com ele”.

Um caso de uso é a descrição de uma determinada ação ou comportamento de um sistema que produz um resultado para um determinado agente externo a esse sistema, denominado ator.

(Modelagem de caso de uso é um método, aplicável a qualquer modelo de processo de engenharia de requisitos, e UML é uma ferramenta de apoio).

## Casos de uso e atores

Caso de uso é a descrição de uma sequência de atividades executadas por um agente externo ao sistema sem que sejam revelados detalhes do funcionamento interno ao sistema.

Atores são os agentes externos ao sistema que executam uma determinada ação e que esperam algum resultado.

- Ator é qualquer entidade externa que interage com o sistema: usuários, outros sistemas de software ou até mesmo dispositivos de hardware.
- Um ator nunca é um componente interno do sistema, ou mesmo qualquer detalhe interno ou componente de implementação dele.
- Um sistema nunca é um ator de si mesmo.

## Descrição de casos de uso

Podemos considerar a descrição em linguagem natural, desde que sequencial, como uma descrição de caso de uso

- **Identificação:** nome do caso de uso.
- **Escopo:** descreve sucintamente a que contexto se refere o caso de uso, podendo ser um processo de negócio, um sistema ou um subsistema.
- **Descrição do propósito:** descrição sucinta do objetivo do caso de uso.
- **Ator primário:** ator principal do caso de uso.
- **Pré-condições e pós-condições:** o que deve ser verdadeiro antes e após a finalização do caso de uso.
- **Fluxo normal:** descrição do passo a passo normal do caso de uso.
- **Fluxo alternativo:** descrição de passos alternativos que podem ser executados dentro de uma sequência normal de passos.
- **Requisitos relacionados:** lista de requisitos que estão representados pelo caso de uso. Um caso de uso pode conectar um ou mais requisitos de tipos diferentes, funcionais e não funcionais.

## **Diagrama de caso de uso**

Diagrama de casos de uso é um diagrama da UML que tem por objetivo mostrar, a partir de um ponto de vista estático, o conjunto de casos de uso, atores e seus relacionamentos. Visão estática é o termo usado para descrever uma parte do sistema sob o ponto de vista estrutural.

## **Estrutura do diagrama de caso de uso**

### **Relacionamento entre casos de uso**

Três tipos de relacionamento entre casos de uso: inclusão, extensão e herança.

#### **Inclusão**

Significa que o comportamento definido no caso de uso de inclusão é incorporado ao comportamento do caso de uso base, ou seja, para que este seja executado, obrigatoriamente o caso de uso de inclusão também deverá ser executado. Na notação da UML, um relacionamento de inclusão entre casos de uso é mostrado como uma dependência (seta pontilhada) estereotipada com a palavra-chave include.

#### **Extensão**

Significa que o comportamento definido no caso de uso de extensão pode ou não ser incorporado ao comportamento do caso de uso base, ou seja, para que este seja executado, o caso de uso de extensão pode ou não ser executado, notação da UML, um relacionamento de extensão entre casos de uso é mostrado como uma dependência (seta pontilhada) estereotipada com a palavra-chave extend.

#### **Herança**

Significa que o caso de uso filho herda o comportamento e o significado do caso de uso pai, acrescentando ou mudando seu comportamento, notação da UML,

um relacionamento de herança entre casos de uso é mostrado com uma linha cheia com uma seta aberta.

Podemos concluir que os casos de uso filho são especializações se, e somente se, mantiverem o mesmo objetivo (filho é uma especialização do caso de uso pai, ou o pai é uma generalização do caso de uso filho)

### **Relacionamento entre atores**

Atores possuem apenas um tipo de relacionamento: herança. Significa que o ator filho herda o comportamento e o significado do ator pai, acrescentando ou mudando seu comportamento, um relacionamento de herança entre atores é a mesma notação utilizada para cenários de uso: uma linha cheia com uma seta aberta (UML). Ator Filho é uma especialização do Ator Pai, ou que o Ator Pai é uma generalização do Ator Filho.

## **Modelagem de processos de negócio**

### **Introdução à modelagem de processos de negócio**

Processo de negócio são atividades relacionadas a um determinado negócio, que são executadas em uma determinada sequência e que produzem um determinado resultado ou objetivo

• Why/Por que/Motivo: representa o objetivo do processo. • Where/Onde: representa onde o processo (a atividade) é executado, as características do ambiente. Podemos representar uma unidade da organização ou até mesmo uma localização geográfica. • When/Quando: representa o momento em que a atividade é executada e suas restrições temporais. • Who/Quem: representa quem é o responsável ou os envolvidos na execução da atividade. Se fizermos uma analogia ao modelo de caso de uso, são os nossos atores. • What/O que: representa um conjunto de informações que serão utilizadas na execução da atividade, são os artefatos que serão produzidos e/ou consumidos na atividade. • How/Como: é a junção de todos os 5W's que definem como as atividades serão executadas. parei na

## **O papel do analista de negócio**

O analista de negócio é o profissional responsável por captar as reais necessidades dos stakeholders e não apenas seus desejos expressos. Ele estabelece um elo entre o usuário do sistema de informação e o sistema de informação propriamente dito, estabelece a ligação entre a área de negócio e a área de TI

O analista de negócio atua nas seguintes áreas de conhecimento: • análise de requisitos; • análise corporativa; • avaliação e validação da solução; • elicitação; • gestão e comunicação de requisitos; • planejamento e monitoramento da análise de negócio

## **Regras de negócio**

Elas são um conjunto de restrições que definem como um processo de negócio de uma organização deve ser executado, que, além de representar determinados conhecimentos a respeito de um processo, também representam importantes aspectos restritivos na execução deste processo

exemplos: • O cliente não pode efetuar saques no período entre 23h00min e 05h00min. • O cliente pode efetuar um saque de, no máximo, R\$300,00 em uma única operação. • O limite de saques diários de um cliente é de R\$1000,00. • Caso o cliente efetue um saque com opção de contratação de empréstimo, serão cobrados juros de 10% ao mês.

## **UML**

### **Diagrama de processo**

• Recursos: representam os objetos de uma organização, como pessoas, materiais e informações, que são usados, consumidos refinados ou produzidos em um processo de negócio. • Processos: representam atividades executadas em uma organização. • Objetivos: representam o resultado que se deseja alcançar, as metas da organização. • Regras: representam as particularidades que restringem algum aspecto do negócio e representam conhecimento do negócio. • Eventos: representam a mudança de estado do negócio. Um evento pode ser gerado por um

processo, que, inclusive, pode estar fora do negócio e é recebido por um ou mais processos.

### **Processo**

O símbolo que representa um processo pode possuir ou não a indicação do estereótipo <>, mas obrigatoriamente deverá ser representada a descrição textual do processo.

### **Objeto**

de negócio Um objeto de negócio é a representação de uma entrada ou saída de um processo. Em geral, são utilizados estereótipos para diferenciar e explicitar melhor os tipos de objeto de negócio, que são: <>

### **Objeto de informação**

Objeto que identifica informações produzidas ou consumidas pelo processo

### **Dependência**

A conexão utilizando linha tracejada e seta indica dependência entre os componentes do modelo, sendo que a direção da seta indica a direção da dependência.

diagramas utilizados na modelagem de processos de negócio: • Diagrama de caso de uso. • Diagrama de modelo de processo. • Diagrama de atividade.

### **Diagrama de atividade**

Definido na UML, o diagrama de atividade representa um fluxo de atividades que tem como objetivo atingir um determinado objetivo.

**Adornos da UML =>** Adornos são mecanismos ou elementos da linguagem UML utilizados com o único intuito de facilitar o entendimento ou ratificar o significado de algo que estamos querendo representar.

## **UNIDADE III**

## AS VISÕES DA UML

Um sistema de software pode ser organizado em cinco visões e cada visão possui um conjunto de diagramas UML que representam aspectos particulares desse sistema.

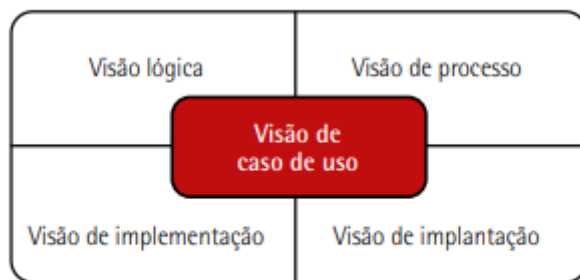


Figura 43 – Visões da UML

### Visão de caso de uso

Tem como objetivo capturar as funcionalidades, os requisitos e seu comportamento sob a ótica do usuário final, ou dos atores.

### Visão lógica

Ou visão de projeto ou visão de classe, tem como objetivo representar como as funcionalidades serão implementadas sob o aspecto da solução de projeto. Representa a estrutura estática de um sistema, seus componentes e o relacionamento entre eles e como esses interagem para resolver um determinado problema

### Visão de processo

Também chamada de visão de concorrência, a visão de processo tem como objetivo capturar aspectos de paralelismo de execução de atividades sob o ponto de vista não funcional de um sistema.

### Visão de implementação

Também chamada de visão de componentes, a visão de implementação tem como objetivo representar aspectos físicos necessários para a construção do sistema e como eles interagem e fazem interface com o sistema.

## **Visão de implantação**

Também chamada de visão de organização, tem como objetivo representar a organização física de hardware do sistema, como computadores, servidores e periféricos e como eles se relacionam com o sistema.

## **VISÃO ESTRUTURAL ESTÁTICA E DINÂMICA**

### **Visão estrutural estática**

Visão de caso de uso produz modelos que representam as necessidades do negócio, que são traduzidas nas funcionalidades de um sistema de software.

É na fase de projetos, também chamada de fase de design, que definimos como os problemas do sistema de software serão resolvidos e como as funcionalidades serão implementadas.

### **Conceitos fundamentais da orientação a objetos**

Podemos dizer que o paradigma da orientação a objetos é uma forma de se desenvolver um sistema. Nesse paradigma o software é visto como um conjunto de componentes que interagem entre si para resolver um determinado problema, a cada componente dá-se o nome de objeto.

A orientação a objetos é uma técnica para modelar sistemas que tem por objetivo a diminuição da diferença semântica entre a realidade do negócio, que está sendo modelado, e os modelos construídos para a solução dos problemas deste mesmo negócio.

### **Classes**

Podemos definir classe, ou classe de objetos, como um grupo de objetos com mesmas propriedades (atributos), comportamento (operações), relacionamentos e semântica.

### **Objeto**



Podemos afirmar que todo objeto possui uma identidade, representada por um conjunto de informações conferidas aos seus atributos, e que todo objeto possui um estado

### **Encapsulamento**

Encapsulamento significa deixar visível, ou deixar acessível a outros objetos, apenas o que é necessário.

### **Abstração**

Abstração está ligada à nossa capacidade de selecionar determinados aspectos do problema e isolar o que é importante para algum propósito, ou seja, é dar ênfase àquilo que é necessário.

- Modelo de classe de domínio: desenvolvido na fase de análise, o modelo de classe de domínio representa os objetos, ou classes, inerentes ao domínio do problema que queremos resolver

- Modelo de classe de especificação: construído na fase de desenvolvimento, o modelo de classe de especificação adiciona ao modelo de classes de domínio, objetos ou classes específicos para a solução do problema sob o aspecto tecnológico

- Modelo de classe de implementação: o modelo de implementação nada mais é que a implementação das classes, especificadas no modelo de especificação, construídas ou codificadas em alguma linguagem de desenvolvimento orientada a objetos

### **Modelo de classes de domínio**

Modelo de classes de domínio representa os objetos, ou classes, inerentes ao domínio do problema que queremos resolver

### **Conceitos fundamentais do modelo de domínio**

Como vimos, um objeto é composto por atributos e métodos, e uma classe pode ser considerada como a especificação de um objeto.

Normalmente identificamos uma classe pelo que ela representa dentro do domínio do negócio em que estamos trabalhando, por exemplo: a classe Cliente Pessoa Física representa todos os clientes do tipo pessoa física que podem efetuar saques em nosso terminal de autoatendimento.

Os métodos de uma classe são representados, na UML, abaixo da identificação de atributo

Um objeto possui um conjunto de informações que os caracterizam, chamados atributos. Quando damos valores a esses atributos, esse objeto passa a ter uma identidade.

Na UML, um objeto é representado da mesma forma que representamos uma classe, porém com o nome da classe sublinhado.

### **Relacionamento entre classes**

Existem cinco tipos de relacionamento entre objetos. São eles: dependência, associação, agregação, composição e a herança

#### **Dependência**

Dependência é um relacionamento em que um objeto depende de informações de outro objeto para a execução de um determinado comportamento. Na UML, representamos a dependência utilizando uma seta tracejada

#### **Associação**

Mostra que um objeto pode se relacionar com outro, que existe uma conexão entre esses objetos. Existem dois tipos de associação: • Associação binária: amplamente utilizada, é a associação entre duas classes apenas. • Associação enésima: raramente utilizada, é a associação entre mais de duas classes

#### **Agregação**

A agregação é utilizada para representar uma conexão entre dois objetos, sendo que essa conexão define uma relação todo-parte entre esses objetos, ou seja, um objeto está contido no outro (Bezerra, 2006). Na UML, representamos agregação utilizando uma reta saindo da classe que representa a parte e se conectando a um losango, também chamado de diamante, na classe que representa o todo.

## **Composição**

A composição tem exatamente o mesmo conceito da agregação, ou seja, estabelece uma relação todo-parte entre dois objetos. A única diferença entre composição e agregação é que a classe que representa a parte da relação, para existir, depende da classe que representa o todo, ou seja, o ciclo de vida do objeto da classe parte depende do ciclo de vida do objeto da classe todo. Na UML, representamos a composição de forma semelhante à agregação, utilizando uma reta saindo da classe que representa a parte e se conectando a um losango na classe que representa o todo. Todavia, na composição, o losango, ou diamante, é preenchido.

## **Classes associativas**

São classes que, em vez de estarem ligadas à outras classes, estão ligadas a uma associação. Normalmente utilizamos classes associativas quando desejamos armazenar informações importantes de uma associação. Na UML, representamos uma classe associativa da mesma forma que representamos uma classe “normal”, contudo, ela é ligada por uma linha tracejada

## **Herança**

Na orientação a objetos, assim como no mundo real, um objeto pode herdar características e comportamentos de outro objeto. Traduzindo para a terminologia correta, um objeto pode herdar atributos e métodos de outro objeto.

Um objeto é uma instância de uma classe ou uma classe é uma especificação de um objeto, uma classe herda atributos e métodos de outra. Nessa situação temos dois papéis de classes na relação: classe mãe e classe filha. Na UML, representamos herança com uma seta direcional, sendo que a ponta da seta é cheia

Todavia, existe ainda outro tipo de herança: a herança múltipla. Diferentemente da herança simples, na herança múltipla uma classe filha pode herdar atributos e métodos de duas ou mais classes mãe

## **Diagrama de classes**

Diagrama de classes é um diagrama UML que tem como objetivo representar a estrutura estática das classes de um sistema de software

## **Diagrama de objetos**

O objetivo de um diagrama de objetos é representar instâncias de classes, ou seja, objetos e suas respectivas ligações ou associações. O diagrama de objetos fornece uma visão dos objetos, suas informações e seus relacionamentos

## **Visão estrutural dinâmica**

Um sistema de software, dentro do paradigma da orientação a objetos, pode ser considerado como uma coleção de objetos que possuem atributos e métodos e que interagem entre si para resolver um determinado problema. Já vimos que o modelo estrutural estático, representa a coleção desses objetos, seus atributos, métodos e seus relacionamentos, mas a forma como eles interagem entre si para resolver um problema não pode ser visualizada em um diagrama de classes.

Visão estrutural dinâmica, também chamada de visão comportamental, tem como objetivo representar a interação dos objetos para atingir um determinado objetivo.

## **Realização de casos de uso**

Em orientação a objetos definimos as responsabilidades de cada objeto. A abordagem para definição de responsabilidades de objetos é chamada de método dirigido a responsabilidades.

## **Classes de análise**

A responsabilidade de um objeto é a “obrigação” que ele deve cumprir no sistema de software do qual faz parte, se o objeto não consegue cumprir com a sua “obrigação” de forma autônoma, precisando da colaboração de outros objetos.

Os objetos são divididos e categorizados em três grupos de acordo com seu tipo de responsabilidade: classe entidade, classe de controle e classe de fronteira. A forma de organizar essas classes, também chamadas de classes de análise, vai ao encontro de um dos princípios fundamentais da orientação a objetos: divisão de responsabilidades.

## **Entidade**

Uma entidade, classe de entidade ou ainda objeto de entidade, são objetos mais próximos do domínio do mundo real que o sistema representa. Esses objetos têm

como objetivo principal manter informações referentes ao domínio de problema que queremos resolver.

### **Fronteira**

Têm como responsabilidade dividir o ambiente interno do sistema e suas interações externas.

### **Controle**

Classes de controle, objetos de controle ou ainda controladores são objetos que têm como objetivo realizar o sequenciamento da execução de um caso de uso na estrutura de objetos do sistema, fazer a coordenação entre as camadas internas do sistema, representadas pelas classes de entidade, com as camadas externas ao sistema, representadas pelas classes de fronteira.

### **Comportamento de objetos**

Métodos têm como objetivo a realização de alguma atividade ou algum comportamento por parte do objeto. Comportamentos, ou métodos de objetos, possuem certos aspectos a serem conceituados, como a forma que os representamos e características próprias da orientação a objetos que veremos na sequência.

### **Polimorfismo**

Polimorfismo é quando um objeto tem um comportamento diferente para a mesma ação.

Polimorfismo é quando uma classe filha herda um método de uma classe mãe, ou seja, o método na classe filha possui a mesma assinatura do método da classe mãe, todavia possuem implementações diferentes, reagem de maneiras diferentes à mesma ação. Polimorfismos nos conferem duas características importantes na orientação a objetos:

- Facilidade na introdução de novas implementações, possibilitando manutenibilidade e extensão ao sistema.
- Encapsulamento das variações de implementação.

### **Visibilidade de atributos e métodos**

encapsulamento é a capacidade que um objeto tem de “esconder” de outros objetos que se relacionam com ele detalhes internos, como detalhes da implementação de um método.

Visibilidade indica quando e em que nível um atributo ou um método de um objeto pode ser acessível aos objetos que se relacionam com ele. Em orientação a objetos, temos três níveis de visibilidade de atributos e métodos:

- Público: o atributo ou o método pode ser acessado por qualquer classe. Na UML, indicamos que um atributo ou um método é público utilizando o sinal + (positivo)
- Privado: um atributo ou método privado pode ser acessado somente na própria classe em que está declarado. Na UML, indicamos que um atributo ou um método é privado utilizando o sinal – (negativo)
- Protegido: um atributo ou um método protegido pode ser acessado apenas na classe em que está declarado e em suas classes filhas. Na UML, indicamos que um atributo ou um método é protegido utilizando o sinal #

## **Interfaces**

Interface é um contrato que contém apenas assinaturas de métodos (não contém atributos) e que podem ser implementados por mais de uma classe. Na UML, representamos interfaces com o estereótipo <<interface>>

## **Ciclo de vida de objetos**

Sistema de software, um objeto é criado, utilizado e destruído, obrigatoriamente nessa sequência. Isso é o que chamamos de ciclo de vida de objetos.

Primeiro há a criação, ou a construção, de um objeto. Construir um objeto significa dar identidade a um objeto ou criar uma instância de uma classe.

Construtor é um método da própria classe que tem como objetivo a criação da estrutura do objeto em memória.

- Um construtor é um método da própria classe.
- Um construtor é um método público.
- Um construtor não possui saída.
- Um construtor pode ou não receber parâmetros de entrada.

Se para construir um objeto temos o construtor, para destruir o objeto temos o destrutor, que tem como objetivo remover o objeto da memória. Assim como o construtor, o destrutor também é um método da própria classe.

O destrutor obrigatoriamente não possui saída e não possui parâmetros de entrada. Na UML o destrutor é representado da mesma forma que o construtor, porém acrescido de um ~ (til)

### **Comunicação entre objetos**

A comunicação entre objetos se dá pela chamada de métodos. Para isso, é fundamental os conceitos de encapsulamento e visibilidade de métodos.

### **Mensagem**

Para que um objeto execute um método é necessário enviar a este objeto uma mensagem solicitando a execução do método em questão.

Uma mensagem pode ter três significados:

- Chamada de função ou procedimento: Significa que um objeto está solicitando a execução de um método de outro objeto.
- Envio explícito de mensagem utilizando um tipo de serviço bem específico e especializado para envio e recebimento de mensagens
- Evento: quando uma mensagem é enviada para um objeto do sistema originária de um evento externo ao sistema.

### **Diagrama de sequência**

O diagrama de sequência da UML representa a interação de um conjunto de objetos, a troca de mensagens entre eles para resolver um problema específico. Uma característica positiva do diagrama de sequência é que podemos visualizar a troca de mensagens de forma sequencial e encaixada em uma linha de tempo.

### **Mensagens síncronas**

Criam uma dependência do estado do objeto que enviou uma mensagem com o estado do objeto que a recebe. Isso significa que o objeto que enviou a mensagem

não tem seu estado alterado, não executa nenhuma ação, até que o objeto que recebeu a mensagem permita.

### **Mensagens assíncronas**

Ao contrário das mensagens síncronas, as assíncronas não criam uma dependência do estado do objeto que enviou a mensagem com o estado do objeto que a recebe. Ou seja, o objeto que enviou a mensagem pode executar qualquer ação independentemente da reação do objeto que recebeu a mensagem.

### **Criação e destruição de objetos**

Como vimos no ciclo de vida de um objeto, existem dois métodos, construtores e destrutores, que têm como objetivo criar e remover um objeto da estrutura de memória de um sistema.

### **Autodelegação de mensagens**

Métodos públicos são acessíveis por qualquer objeto no modelo, logo, são passíveis de envio de mensagens. Um método privado de um objeto não pode ser acessado por outros objetos, apenas pelo objeto na qual está declarado, logo, esse método não é passível de receber mensagens de outros objetos. Todavia, um objeto pode enviar uma mensagem para ele mesmo, solicitando a execução de um método privado. Esse movimento é chamado de autodelegação

### **Notação de estruturas de decisão e de repetição**

A interpretação é a mesma que damos em lógica de programação, ou seja: em estruturas de decisão, uma mensagem só é enviada para um objeto se uma determinada condição for satisfeita e, em um laço de repetição, uma mensagem é enviada sucessivas vezes, tantas quantas forem especificadas no laço.

### **Estereótipos de classes de análise**

Como vimos em classes de análise, as classes podem ser classificadas e organizadas de acordo com a responsabilidade que possuem dentro do modelo: entidade, controle e fronteira.



# UNIDADE IV

## PROJETOS DE DADOS E CLASSES E PROJETO ARQUITETURAL

### Projeto de dados e classes

A fase de projetos é organizada em quatro fases: projeto de dados, projeto arquitetural, projeto de interfaces e projeto de componentes.

A fase de projetos de dados tem como objetivo a geração do modelo de dados e a transformação de classes e objetos conceituais em classes e objetos equivalentes em projeto.

### Introdução ao projeto de dados

O projeto de dados, ou modelagem de dados, tem como objetivo definir uma estrutura de informações necessárias para implementar o sistema de software, devemos determinar a organização, métodos de acesso, associações e alternativas de processamento dessas informações.

boa parte dessas atividades é feita pelo SGBD, que é o responsável por prover os mecanismos de acesso, gerenciamento e processamento das informações em um repositório de dados

O processo de projeto de banco de dados pode ser subdividido em três fases: projeto conceitual, projeto lógico e projeto físico

- Projeto conceitual: tem como objetivo produzir um modelo de alto nível, ou seja, sem detalhes específicos de um SGBD, que represente os requisitos de dados. Esses requisitos expressam tudo aquilo que deverá ser armazenado pelo sistema
- Projeto lógico: essa fase do projeto utiliza como base o modelo produzido no projeto conceitual e monta o modelo de dados de implementação
- Projeto físico: é a implementação do modelo lógico em uma linguagem de programação que possa fazer a transação do modelo lógico para o esquema interno do SGBD.

## **Introdução a banco de dados relacionais**

O modelo entidade relacional enxerga os dados do mundo real como o conjunto: entidade, atributos e relacionamento. Cada entidade, ou um conjunto de entidades, gera uma tabela, seus atributos ou características são representados por colunas desta tabela e cada linha desta tabela representa uma instância dessa entidade.

## **Bancos de dados relacionais versus orientação a objetos**

Pode-se notar que o modelo relacional e o modelo orientado a objetos diferem: no modelo relacional, não temos o armazenamento dos métodos que poderiam agir sobre uma determinada classe.

Semelhanças:

- Assim como na orientação a objetos, uma entidade possui atributos.
- Assim como na orientação a objetos, uma entidade possui uma identidade, que pode ser obtida a partir de um conjunto de valores de seus atributos. Podemos dizer então que cada linha de uma tabela é uma instância de uma entidade, ou um objeto.
- Assim como na orientação a objetos, as entidades se relacionam.

## **Projeto arquitetural**

Qual o papel da arquitetura?

Na engenharia de software, o papel da arquitetura é projetar estruturas, a constituição de um software ou a contextura de um todo

## **O que é arquitetura de software?**

- Arquitetura de software é uma representação do sistema que auxilia na compreensão de como ele irá se comportar
  - Arquitetura de software é uma descrição de como um sistema de software é organizado
- Arquitetura é a organização fundamental de um sistema incorporada em seus componentes
- Arquitetura de software pode ser definida como uma representação, em alto nível de abstração, dos componentes de um sistema de software

Está claro que a arquitetura de um software não é o software em si, mas sim a representação, o modelo, a organização do software que será efetivamente construído.

### **A importância da arquitetura de software**

A arquitetura de software produz modelos e organiza a estrutura de um software de tal forma que sirva como o principal guia para a construção. A arquitetura de um sistema de software serve como modelo tanto para o projeto quanto para a construção, facilitando inclusive a divisão das tarefas a serem realizadas pela equipe de projeto e de construção. Além do mais, a arquitetura é o principal meio para o atingimento de qualidade de um sistema como: desempenho, manutenibilidade e segurança.

Importância da arquitetura de software:

- Facilita a comunicação entre os envolvidos no projeto. •

Representa, em escala menor e compreensível, a forma pela qual os componentes de um sistema são estruturados e interagem.

- Modela e coloca em evidência as decisões de projeto como fator fundamental para o sucesso do sistema de software.

### **Processo de arquitetura de software**

participação de arquitetura de software pode ser dividida em cinco fases, e deve ser iniciada antes mesmo da fase de projetos do ciclo de vida da engenharia de software:

| Fases do ciclo de vida da engenharia de software |   | Fases da arquitetura       |
|--------------------------------------------------|---|----------------------------|
| Análise de requisitos                            | ➡ | Pré-projeto                |
| Projeto                                          | ➡ | Análise de domínio         |
|                                                  |   | Projeto esquemático        |
| Implementação e testes                           | ➡ | Desenvolvimento de projeto |
|                                                  |   | Construção                 |
| Implantação                                      | ➡ | (sem correspondência)      |

## **Processo de arquitetura de software em aspectos humanos**

existem alguns interessados no processo de definição de arquitetura de um sistema de software: analistas de requisitos, desenvolvedores, gerentes de projetos e arquitetos de software

arquiteto tem a obrigação de:

- Conhecer os aspectos culturais do seu cliente.
- Conhecer as estratégias de mercado do cliente e o que ele pretende com software.
- Conhecer as limitações de tecnologia e infraestrutura do cliente.
- Conhecer o domínio do negócio e do problema sobre o qual pretende promover uma solução.

o arquiteto tem a obrigação de:

- Manter-se constantemente atualizado a respeito das metodologias, tecnologias, soluções e plataformas de desenvolvimento.
- Conhecimento nível sênior em engenharia de software, que compreende conhecimento em pessoas, ferramentas e processos inerentes à engenharia de software.

## **VISÕES DA ARQUITETURA DE SOFTWARE**

A arquitetura de um sistema de software define os componentes que formarão este sistema de software. Estes componentes podem ter o menor nível de composição, dependendo do nível de abstração utilizado nessa composição.

- A visão estática da arquitetura promove a visão da organização dos componentes do software e de suas relações com elementos de dados (banco de dados, arquivos-texto etc.) com sistemas de hardware e com outros sistemas de software.
- A visão dinâmica da arquitetura promove a visão comportamental do sistema e de seus componentes durante a execução de uma determinada ação. Nessa visão, definimos como os componentes do sistema reagem a eventos internos e externos e ainda a forma como eles se comunicam entre si, ou seja, o protocolo de comunicação interno, e até mesmo como eles se comunicam com componentes externos ao domínio do software, chamado de protocolo de comunicação externa.

## Utilização de padrões na arquitetura de software

Padrões são determinadas soluções que comprovadamente funcionam para resolver um determinado problema

Padrão => uma solução reutilizável descrita com base em três partes: um contexto, um problema e uma solução. • Contexto: estende o problema a ser solucionado, apresentando situações de ocorrência desses problemas. • Problema: determinado por um sistema de forças, em que estas forças estabelecem os aspectos do problema que devem ser considerados. • Solução: mostra como resolver o problema recorrente e como balancear as forças associadas a ele.

## Estilo arquitetural

Estilo arquitetural, modelo arquitetural ou ainda padrão arquitetural é a organização, em um alto nível de abstração, de um sistema de software em um conjunto finito de subsistemas. Essa organização especifica as responsabilidades, regras de organização e o relacionamento entre estes subsistemas

## Estruturação de sistemas em subsistemas e camadas

Imaginemos então que pudéssemos separar em uma "caixa" todas as classes que fossem relacionadas a páginas *web*, em uma segunda "caixa" todas as classes que executassem algum tipo de regra de negócio e em uma terceira todas as classes que fizessem interface com o banco de dados, e que essas "caixas" tivessem um protocolo de comunicação entre elas, como mostra a figura a seguir:

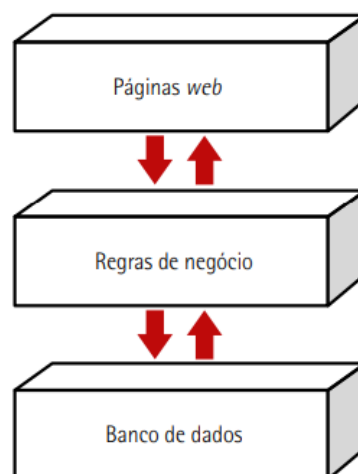


Figura 102 – Organização em "caixas"

- Divisão de responsabilidades: cada “caixa” teria uma responsabilidade bem-definida.
- Baixo acoplamento: as “caixas” teriam apenas as dependências necessárias
- Boa manutenibilidade: agora é possível saber exatamente em que “caixa” se encontram os componentes responsáveis por fazer algum tipo de cálculo ou executar alguma regra de negócio
- Reúso: os componentes podem ser reutilizáveis

### **Introdução a padrões de projeto (design patterns)**

Padrão de projeto, ou design pattern, é a menor estrutura arquitetural proveniente dos subsistemas de um sistema de software e do relacionamento entre eles.

A maneira mais simples de entender e assimilar design pattern é buscar um padrão dentro deste catálogo, entender qual é seu objetivo, ler o seu modelo de classes e implementar, na forma de prova de conceito, na linguagem orientada a objetos de nossa preferência.

**Visão dinâmica =>** objetivo principal da visão dinâmica da arquitetura é representar a realização dos casos de uso, que fundamentalmente se dá pela troca de mensagens entre os objetos no decorrer do tempo

**Documentação de arquitetura =>** sistema de software é um conjunto de componentes dispostos em uma estrutura, que diz como eles estão arranjados, suas composições, dependências e ainda como eles se relacionam para resolver um determinado problema, além de definir como estes componentes estão distribuídos em uma infraestrutura

### **Três as visões arquiteturais:**

- Visão modular: representa a visão do sistema em termos de unidade de implementação; essas unidades podem ser classes, componentes ou módulos.
- Visão componente e conector: representa a forma pela qual os componentes interagem, ou seja, seus protocolos de comunicação.

- Visão de alocação: representa a forma pela qual esses componentes estão distribuídos em uma infraestrutura.

Visão modular => Diagrama de pacotes / Diagrama de classes

Visão componente e conector => Diagrama de componentes

Visão de alocação => Diagrama de distribuição

Os diagramas a seguir são utilizados para documentar a visão dinâmica de uma arquitetura:

- Diagrama de sequência. • Diagrama de colaboração. • Diagrama de máquinas de estado